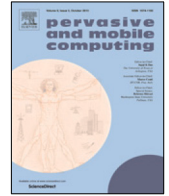




Contents lists available at ScienceDirect

Pervasive and Mobile Computing

journal homepage: www.elsevier.com/locate/pmc

Porting deep neural networks on the edge via dynamic K-means compression: A case study of plant disease detection

Fabrizio De Vita^{a,*}, Giorgio Nocera^a, Dario Bruneo^a, Valeria Tomaselli^b,
Davide Giacalone^b, Sajal K. Das^c

^a Department of Engineering, University of Messina, Messina, Italy

^b STMicroelectronics, Catania, Italy

^c Department of Computer Science, Missouri University of Science and Technology, Rolla, USA

ARTICLE INFO

Article history:

Received 29 January 2021

Received in revised form 4 May 2021

Accepted 11 June 2021

Available online 17 June 2021

Keywords:

ICPS

Compression

Deep learning

Edge

Smart agriculture

ABSTRACT

Cyber Physical Systems (CPS) totally revolutionized the way we interact with the world providing useful services that can support the human being in many aspects of his life. Artificial Intelligence (AI) is another important player for bringing intelligence to CPS and allows the realization of Intelligent Cyber Physical Systems where smart applications can run. However, the constrained hardware of these devices in terms of memory and computing power makes challenging the deployment and execution of powerful algorithms (e.g., deep neural networks). To address this problem, modern solutions involve the use of compression techniques to reduce the memory footprint of deep learning models while saving the accuracy performance. The proposed work focuses on plant disease detection which represents one of the biggest challenges in smart agriculture; in such a context, the possibility to perform a timely diagnosis on crops suspected to be infected can avoid the spread of diseases, thus saving a lot of time and money during the plantation works. In this paper, we realized an intelligent CPS on top of which we implemented an AI application, called *Deep Leaf* that exploits Convolutional Neural Networks to detect the main biotic stresses affecting crops. To meet the hardware requirements of the Edge device running our application, we propose a novel dynamic compression algorithm based on K-Means for the reduction of models footprint. Experimental results show that our detector is able to correctly classify the plant health condition with an accuracy of 95% and demonstrate the effectiveness of the proposed compression algorithm which is able to maintain the same accuracy of the original 32 bit float model, with an overall memory size reduction of about 85.2%.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

The Internet of Things (IoT) widespread catalyzed the evolution of traditional agriculture. Thanks to the ICT advancement, this science today became “smart”, benefiting from the use of tools that allow to improve the productivity and effectiveness of plantation works [1].

In the recent period, plant disease detection gained a lot of interest in smart agriculture. In this sense, a timely detection of those plants suspected to be infected is crucial to avoid the spread of diseases in plantations which can have very high

* Corresponding author.

E-mail addresses: fdevita@unime.it (F. De Vita), giorgio.nocera@studenti.unime.it (G. Nocera), dbruneo@unime.it (D. Bruneo), valeria.tomaselli@st.com (V. Tomaselli), davide.giacalone@st.com (D. Giacalone), sdas@mst.edu (S.K. Das).

costs in terms of time and money. Today, the detection of plants health is one of the biggest challenges in this area because of the large number of diseases that can affect these organisms [2]. Evidently, such a challenge becomes even more severe in plantations where the types of plants can be very large, thus requiring the implementations of effective monitoring systems. Such systems usually integrate Internet communication and computing capabilities into the physical world, exposing also sensing and actuating functionalities, and for this reason are called Cyber Physical Systems (CPS) [3]. Traditionally, the Cloud computing paradigm has played a key role for the realization of CPS providing large storage and computing capabilities. However, modern services and applications pose requirements (i.e., latency and connection stability) that the Cloud is not able to handle. As a result of this condition, the Edge computing paradigm tries to address these new requirements by shifting the computation near to where the data is generated [4] in order to reduce the latency and improve the hardware resource utilization. The shift of computation towards the Edge however comes with a set of challenges which are mainly related to the hardware constraints of these devices which pose significant limitations on the type of tasks they can accomplish.

In such a context, Artificial Intelligence (AI) is an important player for bringing the intelligence to CPS and allows the realization of new types of systems e.g., Intelligent Cyber Physical Systems (ICPS). Recently, a growing interest has emerged towards this technology with the goal to make these systems able to perform “reasonings”. Thanks to machine learning in fact, it is possible to introduce a “cognition” layer that makes an ICPS able to learn a task exploiting its sensing and actuating capabilities [5]. If on the one hand the use of AI brings applications and services to a whole new level, on the other, the limited computing power of Edge devices makes the execution of complex and onerous algorithms very challenging to run effectively. Deep Neural Networks (DNNs) represent a perfect example of this trend. In this sense, they allow to accomplish very complex tasks (e.g., computer vision applications, natural language processing, etc.), but require a large number of parameters (in the order of billions) to perform them, which cause the “explosion” of their memory occupation.

Modern solutions involve the use of compression and quantization techniques to reduce the memory footprint and complexity of a machine learning model in order to fit the constrained hardware requirements of an Edge device, while maintaining a good level of performance [6]. Methods like pruning, weights sharing, and quantization represent the most used approaches to reduce the complexity of a DNN, however, they require a careful tuning or a model re-training that can be very time consuming procedures. In such a context emerges the necessity of developing algorithms with minimal hyperparameters setup that should be performed without retraining the DNNs.

In this paper, we extend the work done in [7] by proposing an improved version of an ICPS solution for smart agriculture capable of detecting the “health” state of plants by exploiting deep learning techniques, using it as a case study of the proposed dynamic compression algorithm. With respect to its previous version, we enabled our plant disease detection application (called Deep Leaf) to recognize with a high level of accuracy a wider range of diseases on a large number of plant types. To do that, we designed a new Convolutional Neural Network (CNN) that allowed to strongly improve the performance of our system. Unfortunately, a more complex and powerful network corresponds also to a larger model size which becomes unsuitable for the hardware limitations of Edge devices.

To address these hardware constraints, we introduce a novel dynamic DNN compression algorithm based on K-Means (KM) for the reduction of the model footprint, while saving the accuracy performance. Given a trained DNN model, the proposed algorithm groups the weights in a set of clusters with the goal of reducing the model complexity and removes the redundant components, thus making it suitable to be executed on a constrained hardware device. We conducted an extensive campaign of experiments to quantitatively analyze the performance of the proposed AI application in terms of accuracy, inference time, memory utilization, and energy consumption. We also show the effectiveness of the proposed dynamic compression method in terms of model size reduction, by comparing it with the compression algorithm available in the X-CUBE-AI tool [8] and an 8 bit quantization technique proposed by TensorFlow lite. The paper contributions are manifold. (i) We propose a novel dynamic compression algorithm based on KM to reduce the memory footprint of DNN models. (ii) We realized an ICPS solution for smart agriculture which is capable of detecting a wide range of diseases on a large number of plants. (iii) We conduct extensive experiments to quantitatively analyze the performance of the proposed ICPS application in terms of accuracy, inference time, memory utilization, and energy consumption, showing also the effectiveness of the proposed compression algorithm.

The rest of the paper is organized as follows. Section 2 provides a description of the related works with respect to the plant disease detection and model compression techniques available in the literature. Section 3 describes the Deep Leaf software focusing on the new CNN we implemented to recognize the plant health state. In Section 4, we present in detail the proposed dynamic compression algorithm based on KM. Section 5 provides the experimental results derived from testing our software and compression algorithm. Finally, Section 6 concludes the paper with directions of future works.

2. Related work

Deep learning is gaining a lot of interest in the recent period for the realization of smart services on the Edge [9]. A recent survey [10] provides useful insights about how to leverage both these technologies for the realization smart applications in several context domains (e.g., multimedia, transportation, industry, etc.).

The problem related to plant disease detection is not new to the literature, in [11] authors present a survey where they describe several techniques to address this problem. On the other hand, the challenge related to models compression and

acceleration is gaining a lot of interest in the recent period. In this section, we present the existing solutions to address both these problems and we compare them with our proposed approach.

Authors in [12] present several deep learning models for the disease detection and diagnosis. The solutions here presented involve the use of well known CNNs (e.g., AlexNet, GoogLeNet, VGG, etc.) that reach very high level of accuracies (in the order of 99%), however, the use of these models is unaffordable for the hardware of an Edge device both in terms of memory footprint and computing power. Our proposed solution can run on a constrained hardware of an ICPS which makes it suitable to perform a fast on-board analysis of plants during the plantation works. In [13] a CNN solution is presented to detect the occurrence of diseases on jutes. The method here presented reached an overall accuracy of 94% outperforming other methods such as: Support Vector Machines (SVM), Random Forests (RF), and K-Nearest Neighbors (KNN). Even if the authors got very good result, also in this case the CNN has not been deployed on a Edge device, moreover they focused only on one type of plants, thus making the proposed solution very application specific. In this sense, the new version of our Deep Leaf software has been upgraded in order to recognize a large number of diseases on several types of plants. In [14], the authors proposed a plant disease detector based on a RF classifier. They adopted a Histogram of an Oriented Gradient (HOG) for extracting features, which are then used to train the classifier. The accuracy of this classifier was very low (about 70%) which makes the use of this technique not suitable for a real application scenario. On the other hand, our goal is to deliver an AI application that should effectively run on the Edge with a high level of accuracy performance.

In general, from the analysis of plant disease detection state of the art, we noticed that the majority of the solutions today proposed are not meant to be used on the smart Edge since they involve very complex models that do not fit the hardware constraints of these devices. Considering this new trend, where the computation is getting closer to data, authors in [15] propose a pruning technique based framework (called *ThiNet*) to compress and accelerate CNN models during both the training and inference processes. Experimental results show very good performance in terms of accuracy and inference time, however, this technique can be applied only to CNNs since it performs a filter level pruning. Compared to the proposed solution, our algorithm is more generic since it compresses also the fully connected layers which are the mainly used on a wide range of DNNs topologies. Another interesting solution is presented in [16] where authors introduce the concept of the *HashedNets*. From a topological point of view, a HashedNet is a traditional DNN which is trained on a special low cost hash function to enable the weights sharing. The result reported in this work demonstrated that this technique allows to obtain a good model compression, while saving its generalization capabilities. In such a context, the main drawback of the proposed method is represented by the necessity to re-train the model to compress it, thereby making this process quite onerous. Our approach on the other hand is performed directly on a pre-trained model and does not require to re-train it. Authors in [17] present a compression method based on KM to achieve weight sharing in convolutions. Despite the good results, the framework here proposed is applied only to convolutions layers and requires a network re-training to get high level performance. As we already said, our algorithm takes as input a pre-trained model and it is able to reach high level of performance without the need of re-training it.

3. Deep Leaf 2.0

Assuming a scenario where a human operator moves within a plantation looking for infected crops, such a process can be very time consuming especially if we consider the huge number of available plants. In such a context emerges the necessity of developing effective monitoring systems to automate or speed-up the diagnosis process through the use of AI-based techniques. Such techniques require a continuous Internet connection which may not be always guaranteed when working in plantations. To address this connectivity issue, the majority of the solutions today available involve the use of technologies like Long Range (LoRa) to cover wide areas. However, if on the one hand these techniques solve the problem related to the connection coverage, on the other they allow the transmission of very short message payloads (in the order of 200 B) which pose significant limitations in the realization of smart services. In such a scenario, the use of the Cloud computing paradigm becomes ineffective. In this sense, other than a continuous Internet connection, such a technique requires also a huge amount of energy to perform a constant monitoring and diagnosis, thus making this approach energy and time consuming. Therefore, there is a need to implement a system capable of performing on-site early analysis of plants health condition, thus supporting the human operator during the diagnosis process.

Deep Leaf 2.0 has been realized to address the above mentioned problems by performing a preliminary on-board analysis of plants suspected of being infected by a disease, with the aim to avoid the spread of diseases on plantations. In this sense, our system does not require any type of connection and allows a quick diagnosis by exploiting deep learning techniques. However, the deployment of applications on constrained Edge devices can be very challenging. Such a task becomes even more complex with AI applications which require a lot of memory and computing power to run effectively. In such a context, the main challenge is represented by the deployment of a software that should be energy efficient, while maintaining good system performance. Specifically, the application we realized is able to detect the main biotic stresses that affect a plant. In Deep Leaf [7], our detector was able to detect 4 “health” states of coffee leaves, which made it very application specific. In this version (i.e., Deep Leaf 2.0), we improved the AI software through the design of a new CNN to enable the recognition of a wider number of diseases.

Fig. 2 depicts the prototype we created using an STM32F746G-disco board connected to an STM32F4DIS-CAM module, both embedded in a polymethylmethacrylate box equipped with some LEDs which are used to light the leaf while the

Table 1
ICPS prototype implementation costs.

Component	Cost
STM32F746G-disco board	55.13\$
STM32F4DIS-CAM	18.59\$
Power supply	14.24\$
Relay single channel	1.98\$
LEDs	18.44\$
Buttons and cables	2\$
Total	110.38\$

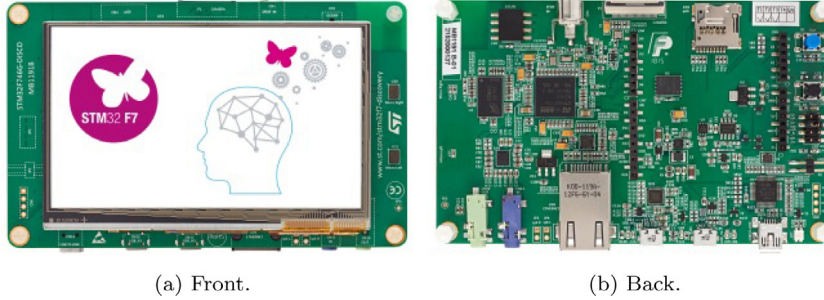


Fig. 1. STM32F746G-disco board.

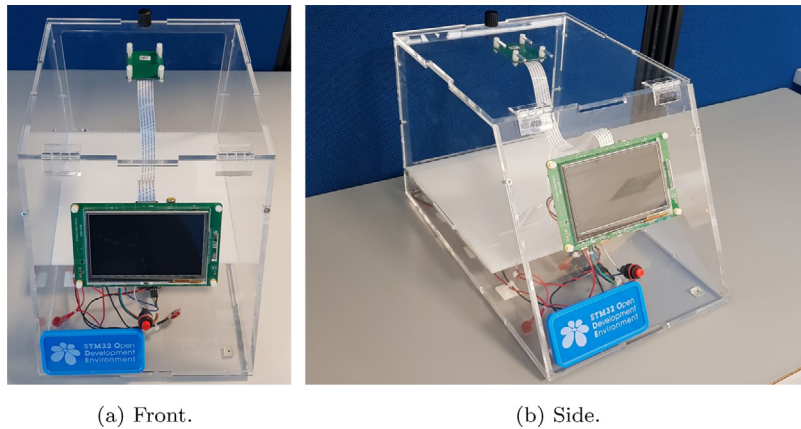


Fig. 2. Deep Leaf 2.0 prototype containing the STM32F746G-disco.

camera module takes the picture. The implementation costs of our ICPS prototype are in the order of 100\$; Table 1 provides a detailed cost description of each component:

however, it is worth to mention that in a large scale production context, the costs for each component would be much lower than our prototype.

The STM32F746GDISCOVERY kit is a complete demonstration and development platform for STMicroelectronics Arm[®] Cortex[®]-M7 core-based STM32F746NG microcontroller. It includes 1 MByte of Flash memory and 340 KBytes of RAM in BGA216 package. The board includes complete peripherals for developing user applications before porting on the custom boards. The Discovery kit enables a diversity of applications by taking advantage of audio, multi-sensor support, AI, graphics, security, video and high-speed connectivity features.

The STM32F746NG microcontroller features include on board peripherals: a 4.3" RGB 480 × 272 color LCD-TFT with capacitive touch screen, two ST-MEMS digital microphones, 128-Mbit Quad-SPI Flash memory, 128-Mbit SDRAM (64 Mbits accessible), two user and reset push-buttons. It also includes connectors to expand the board capabilities: Camera, microSD[™] card, Audio line in and out jacks, 2×USB with Micro-AB, ARDUINO Uno V3 expansion connectors. The on-board ST-Link/V2-1 programmer enables virtual COM port, mass storage and debug port functions. With the power supply, the board can be powered from 3.6 to 12 volts, thus facilitating its integration into existing circuitry. Fig. 1 shows the front and back sides of the board.

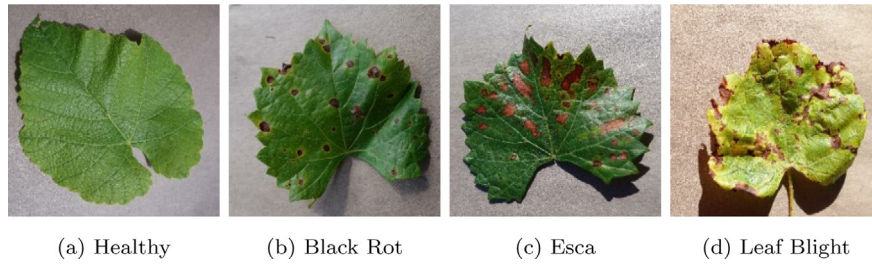


Fig. 3. Example of Grape leaves images.

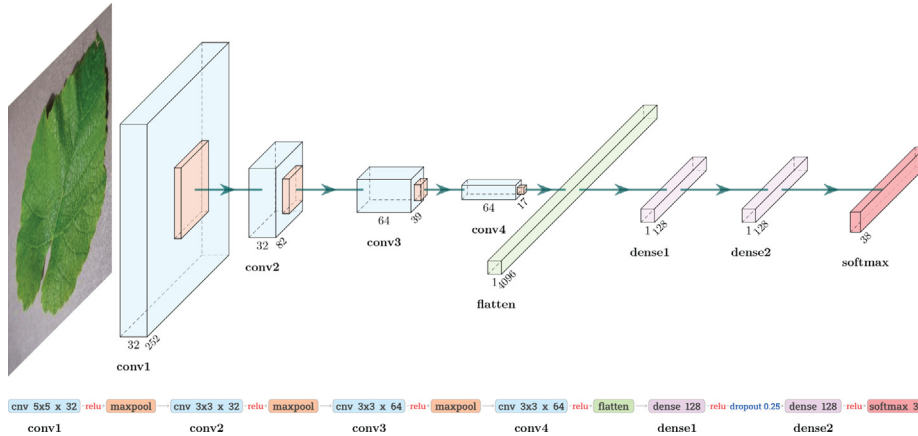


Fig. 4. Complete structure of the CNN.

3.1. Dataset description

To train and test our ICPS system, we used the *Plant Village* image dataset [18], an open access repository based on crowdsourcing to enable the mobile disease detection of plants. Today, the literature proposes several versions of this dataset, however, we used the one available from TensorFlow Datasets (TFDS) [19] consisting of 54,303 RGB healthy and unhealthy leaf images divided into 38 categories by species and diseases. Compared to the previous version of our Deep Leaf software, we upgraded it by designing a new deep learning model capable to detect a larger number of diseases. More specifically, the new dataset is composed by 14 plant species, namely: *Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Bell Pepper, Potato, Raspberry, Squash, Strawberry and Tomato*. Other than healthy plant images, the dataset contains several types biotic stresses e.g., *bacterial, fungal, bugs caused, mold and viral* diseases.

For a better understanding Table 2 reports how the dataset is organized:

Fig. 3 shows an example of 4 images of grapes leaves from the dataset. All the images come with a 1:1 ratio (256×256 pixels) which made possible to avoid the cropping and resizing of the original dataset as the camera module attached to the STM32F746G-disco board captures images with squared ratios. Thanks to the large number of provided samples, it was not necessary to increase the number of images by using augmentation techniques. In this sense, the only preprocessing technique we applied consisted in a normalization process to rescale the pixel values of each channel (i.e., red, green, and blue) from a $[0, 255]$ to a $[0, 1]$ range, thus enabling a faster and better model learning. Finally, we divided the set of images into training, validation and test sets by using the 75% (i.e., 38,012 samples), 15% (i.e., 8146 samples), and 15% (i.e., 8145 samples) of images, respectively.

3.2. CNN description

The problem of classifying the health status of a crop was tackled through the design and development of a multiclass classifier via a supervised approach. The machine learning technique we used to realize the classifier is a DNN belonging to the category of the CNNs. These models are generally used for visual recognition and classification tasks which, differently from classical computer vision systems, automatically learn how to extract the features that allow the classification process. More specifically the structure of the proposed CNN is depicted in Fig. 4.

The CNN network is made up of two parts, namely: the *feature extraction* and the classification one. The first is realized through the use of convolutional layers which are responsible to perform the actual feature extraction and pooling layers

Table 2
Plant Village categories.

Class	Samples
Apple Healthy	1,645
Apple Scab	630
Apple Black Rot	621
Apple Rust	275
Blueberry Healthy	1,502
Cherry Healthy	854
Cherry Powdery Mildew	1,052
Corn Healthy	1,162
Corn Leaf Spot	513
Corn Rust	1,192
Corn Leaf Blight	985
Grape Healthy	423
Grape Black Rot	1,180
Grape Esca	1,383
Grape Leaf Blight	1,076
Orange Haunglongbing	5,507
Peach Healthy	360
Peach Bacterial Spot	2,297
Pepper Healthy	1,477
Pepper Bacterial Spot	997
Potato Healthy	152
Potato Early Blight	1,000
Potato Late Blight	1,000
Raspberry Healthy	371
Soybean Healthy	5,090
Squash Powdery Mildew	1,835
Strawberry Healthy	456
Strawberry Leaf Scorch	1,109
Tomato Bacterial Spot	2,127
Tomato Curl Virus	5,357
Tomato Early Blight	1,000
Tomato Healthy	1,591
Tomato Leaf Mold	952
Tomato Leaf Spot	1,771
Tomato Late Blight	1,908
Tomato Mosaic Virus	373
Tomato Spider Mite	1,676
Tomato Target Spot	1,404
Total	54,303

Table 3
Convolutional layers description.

Layer	Kernel size	Filters	Activation
conv1	5x5	32	<i>ReLU</i>
conv2	3x3	32	<i>ReLU</i>
conv3	3x3	64	<i>ReLU</i>
conv4	3x3	64	<i>ReLU</i>

whose purpose is to reduce the feature map size derived from the convolution operation, other than making the model more invariant to small translations of the input. The latter part is composed by a sequence of fully connected layers that are terminated by a *softmax* layer which allows to determine the most probable class associated with the input image.

From Fig. 4, it is possible to identify the feature extraction part of the network from *conv1* to *conv4* layers. After the feature extraction process, a *flatten* layer is used to transform a multidimensional matrix into a one dimensional vector to be used as input for the classifier part of the network. This part of the network can be identified in layers from *dense1* to *dense2* and the softmax one which provides the actual output of the CNN.

Compared to Deep Leaf [7], in the new model topology we maintained the same number of convolution layers, but we reduced the number of filters by giving a more “linear” structure to the feature extraction part, changing the number of filters from 32, 64, 128, 64 to 32, 32, 64, 64. Such a choice allowed to obtain a smoother feature extraction process without causing the “explosion” of the learnable parameters derived from the use of large filters in the first layers. To address the problem of a too large model size, we also added a pooling layer after each convolution to reduce the number of learnable parameters and the overall network complexity. With respect to the classifier part, we increased the neuron units of the second dense layer (from 32 to 128), thus enabling the learning of a more complex task.

The network was developed to take as input 256×256 pixels RGB images. In order to extract information from these images, the convolutional part of the network was designed with 4 convolutional layers whose details are described in

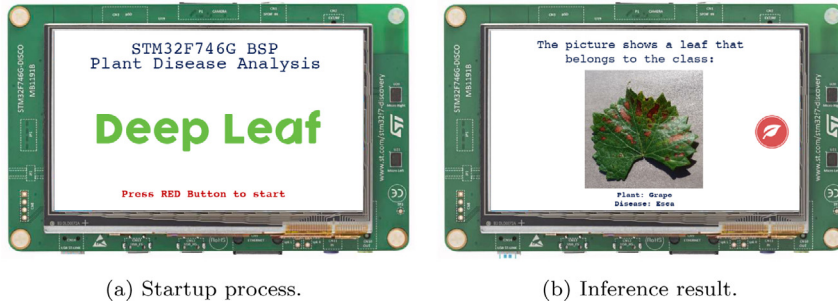


Fig. 5. Deep Leaf application.

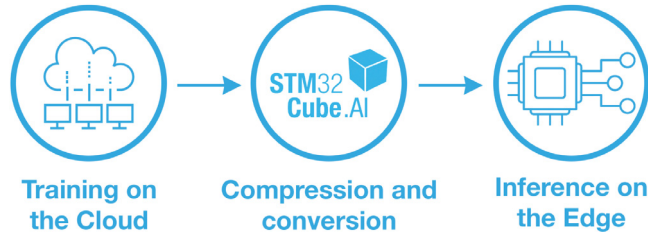


Fig. 6. Workflow steps.

Table 3. A max pooling layer with a *pool size* of 2×2 is inserted after each convolutional one, except for the first one, whose *pool size* is equal to 3×3 .

For the classification part of the network we used three dense layers, two of which are composed by 128 neurons followed by the Rectified Linear Unit (*ReLU*) activation function, and the last one composed by 38 neurons which uses the *softmax* activation function to return the probability of a sample to belong to one of the 38 possible classes. We choose the *ReLU* activation function for each layer of our CNN, because as known by the literature, it allows to speed up the training phase without having significant differences in the accuracy with respect to the other activation functions like the *sigmoid* or the *hyperbolic tangent* [20].

To prevent the network overfitting, we used a dropout layer as a regularization technique with a drop rate of 0.25 between the *dense1* and *dense2* layers. In particular, only during the training phase, the dropout layer at each iteration randomly “shuts down” 25% of the neurons from layer *dense1* by cutting their connections. As a result of this operation, during the training process the weights of these neurons are not updated thus avoiding an overall network over training that could cause the model overfitting. Moreover, it is proven by the literature that the use of this technique makes the system more stable and robust to generalization [21].

To further reduce the possibility to overfit, we used an early stopping technique. Specifically, this technique monitors the loss on the validation dataset and the overall model accuracy saving each time the model which best performed. The training process is automatically stopped if after 10 consecutive epochs (i.e., the “patience” term we set for our model) the model is not able to reduce the loss.

Finally, we used *Adam* optimizer with a *learning rate* of 0.001 and set the maximum number of training epochs to 1000.

3.3. Deep Leaf 2.0 workflow

This subsection presents how we performed the porting of the above described CNN on our ICPS, describing also how Deep Leaf is able to detect the plant health condition. Fig. 6 depicts the main steps of the workflow. The first step consists in the CNN training; because of the hardware limitations of Edge devices, this process is entirely performed in the Cloud that allows to train very complex models in a short amount of time. After this step, the trained model passes through a porting process consisting of two phases, namely: a conversion phase and a compression one which are both performed by the X-CUBE-AI tool developed by ST [8]. The X-CUBE-AI tool significantly extends the feature-set of STM32CubeMX by importing an artificial neural network trained with many of the most popular libraries freely available today, such as Keras, TensorFlow Lite, ONNX, Caffe, Lasagne, or ConvnetJS. In this sense, the tool has the capability to automatically convert a pre-trained neural network, and then integrate an optimized library into the user project providing also a set of metrics to validate the neural network model generated on both PCs and STM32 devices, without having to develop ad-hoc C codes.

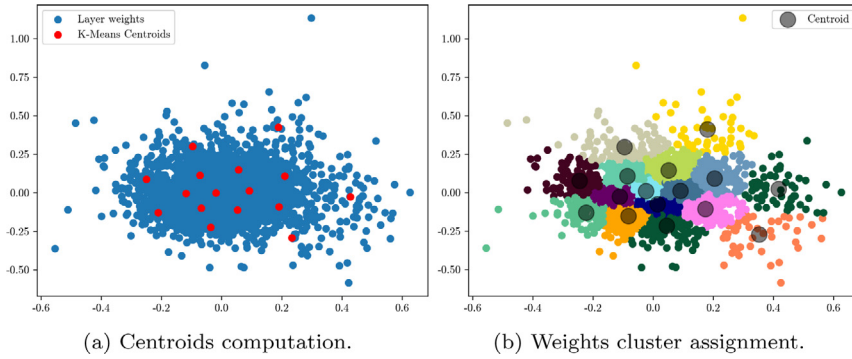


Fig. 7. KM compression using a target-factor equal to 8.

With respect to the compression phase, X-CUBE-AI adopts an algorithm based on the KM clustering that allows to deploy larger networks than STM32 microcontroller resources, storing the weights on external flash memory, and activation buffers on external RAM modules. Integrating a neural network into an embedded system is challenging due to memory occupancy. However, if the model can fit on the embedded memory of a processor, huge energy savings is achieved. The final step of the workflow is the actual inference process performed on the ICPS to detect the health state of plants. At the start of the (microcontroller) board, the GUI asks the user to press the “red button” (Fig. 5(a)) under the display to activate the camera module and the LEDs that help to better focus the subject (i.e., the leaf). Then, the user can insert a leaf inside the box using a slot accessible from the back side (see Fig. 2) and verify its positioning with respect to the camera frame through the display. When the leaf is inside the frame, the user can press on the LCD touch screen to take a picture of the leaf. After this phase, the image is passed as input to the CNN that will perform the inference process. Finally, the classification output is shown to the user in a text form with an icon indicating if the leaf is healthy or not, as shown in Fig. 5(b).

4. A dynamic compression method

DNNs with billions of parameters are becoming increasingly common in AI applications. Such networks are unfortunately not affordable for the hardware restrictions of Edge devices which pose strict limitations both in terms of memory occupation and energy consumption. Today, modern DNN topologies tend to be overparametrized which on the one hand allows to reach good generalization results, even if they do not guarantee a substantial improvement of the accuracy performance [22]. To address this problem, modern approaches involve solutions like quantization to reduce the model complexity. However, the use of such a technique is not always possible because of the huge drop of performance that can derive [23]. As a result of this condition, compression techniques have been proposed as a solution to reduce the models memory footprint while maintaining the same accuracy performance.

In Edge AI, it is common practice to store the weights of convolution and dense layers in flash memory. In most cases, the model training is done with single or double precision float data type, requiring 4 or 8 bytes per value, respectively.

To reduce memory usage in embedded devices, single-precision float data type is typically used, while the model accuracy remains almost unchanged. To further reduce memory requirements, data compression techniques can be used, storing each weight with less than 4 bytes per item. Clustering techniques, based on centroids, are often used for this purpose, because they are very effective for data compression, although their simple implementation.

Specifically, in its current version, the KM compression available in X-CUBE-AI is applied only to fully connected (or dense) layers, because most of the weights of a neural network are concentrated in these layers. Each dense layer is taken into account individually and the weights in it are clustered using the KM algorithm, where the number of centroids is defined as a function of the compression factor we want to achieve. The user may choose a weight compression factor, *target-factor*, equal to x4 or x8. The number of centroids that are generated is:

$$n_{centroids} = 2^{32/(target-factor)}, \quad (1)$$

which leads to 16 or 256 clusters to be automatically identified. Moreover, to mitigate the effect of compressions that can have a negative impact on the model performance, the KM clustering is not applied to the last dense layer representing the output of the network, thus preventing wrong model behaviors. This compression approach has the advantage of being dataset-less, which means that no dataset is requested to apply the compression and optimization algorithms.

Fig. 7 depicts an example of the KM compression with a target-factor set to 8 (i.e., 16 centroids are computed according to Eq. (1)). In such a context, the algorithm performs two steps: it computes the centroids on the data passed as input (see Fig. 7(a)) and assigns each weight to the closest cluster in the euclidean space (as shown in Fig. 7(b)), thus reducing the number of bits necessary to represent the layer.

After the KM clustering, the centroids are saved into a Look Up Table (LUT) (one for each layer), in float format. In particular, the weights of each dense layer are replaced by the index of the relative cluster, whose size can be of 4 or 8 bits according to the selected target-factor. Only the LUT and indexes are saved in the flash memory of the device, so realizing data compression. During the DNN inference stage, the weights are reconstructed by accessing the LUT via the saved indexes.

Even if dense layers contain the majority of weights of a network, when working with CNNs containing a large number of convolutional layers, if not compressed such layers can generate a model footprint which does not fit the hardware requirements of an Edge device. To address this problem, we focused our attention on these type of layers by implementing a new version of the previously described KM compression; in order to improve the effectiveness of the proposed algorithm, we introduce a dynamic compression algorithm according to the layer type.

A convolution layer works very differently respect to a dense one. From a CNN point of view, the convolution step consists in a linear operation between two matrices, usually called *Input* (I) and *Kernel* (K), that produces in output a new matrix called *feature map* (F) [24]. In such a context, the kernel slides over the rows and columns of the input matrix and computes the feature map as follows:

$$F_{i,j} = \sum_h \sum_w I(i+h, j+w)K(h, w), \quad (2)$$

where $F_{i,j}$ is the generic element of the feature map, h is the number of kernel rows (i.e., the height of the kernel), w is the number of kernel columns (i.e., the width of the kernel), i is the row index, and j is the column index. Unlike dense layers, the weights contained in the kernel matrix do not change during this operation, thus enabling the so called *parameters sharing* which is a typical characteristic for these type of layers. As a result, a convolution layer requires a lower number of parameters to be represented because of this property. In this sense, the use of the above described compression method which would consider each convolution layer separately from the others would cause the generation of a large number of redundant centroids (i.e., centroids very close in the euclidean space), with a consequent reduction of the compression effectiveness. To avoid such a condition, in the proposed version of our algorithm, we combine the information coming from the convolutions by merging together all the convolution layer weights and performing the KM clustering on top of them. The advantages of doing so are twofold: we completely remove the presence of redundant centroids, and we strongly reduce the model footprint by remapping all the convolution parameters into a fixed number of clusters. In particular, from our empirical tests, we derived that use of x4 target-factor (i.e., 256 centroids) resulted to be the best trade-off in terms of performance and memory footprint occupation, avoiding too severe compressions that can negatively impact a deep learning model, especially when applied to convolution layers, which are responsible for the feature extraction process.

For a better understanding of the algorithm workflow, we report the dynamic KM compression pseudo code in Algorithm 1.

The algorithm takes as inputs the desired compression target-factor, the dense layers and the convolutional layers data structures containing the weights of the neural network (line 1) and returns the compressed layer weights (line 2). The first lines of the algorithm set the number of centroids to be found on the dense layers (line 3) and initialize two arrays that are used to store the compressed dense and convolution layers weights respectively (lines 4–5). Then, the algorithm iterates over the neural network weights (line 6), it analyzes each layer separately and performs the KM clustering. After the clustering, the algorithm returns the array of centroids $C_{Dense}^{(L)}$ found on the dense layer weights passed as input to the KM (line 7). In such a context, the algorithm iterates over the layer weights (line 8) and computes for each one of them the distance with the centroids $C_{Dense}^{(L)}$. Given the distances, the algorithm computes the argmin value among them (i.e., the closest centroid in the euclidean space), assigns it to a new weight w^* (line 9) and updates the corresponding weight w contained in the *dense* data structure with w^* (line 10). It is worth to mention that the above described procedure is iteratively repeated for every dense layer in the network. Unlike for the dense layers, in the case of convolution layers, the target-factor is kept fixed to x4 (i.e., 256 centroids are computed); as already said, such a choice has been made to mitigate the effect of a too hard compression that would result in an inevitable “failure” of the model. Lines 13–19 are equivalent to what is done in lines 7–11 with the only difference that the KM clustering is performed once on the entire *conv_layers* data structure. Finally, after this process, the algorithm returns the compressed layer weights stored in *dense_layers** and *conv_layers** (line 20).

4.1. Computational complexity analysis

At the beginning of this section we provided a detailed description of the proposed dynamic compression algorithm. In this subsection, we provide its computational complexity analysis. If we do not consider the first lines of the algorithm (lines 3–5) which have an $O(1)$ computational complexity, the core of the algorithm lays in lines 6–19. The external *for loop* in line 6 iterates over the neural network layers and performs the KM algorithm for each dense layer. Lines 7–11 execute the actual KM algorithm whose best known upper bound is $O(s^{kd})$, where k is the number of centroids, d is the samples dimension, and s is the number of samples [25,26]. Assuming that the number of layers in a neural network is much less than the number of weights, we can compute the KM complexity for the dense layers as:

$$O(\|W^{(max)}\|^{(nc)}) \quad (3)$$

Algorithm 1: Dynamic KM compression algorithm

```

1: Inputs:
   • a target-factor determining the number of centroids to be found
   • a dense_layers data structure containing the weights matrix of each dense layer (output excluded)
   • a conv_layers data structure containing the weights matrix of each convolutional layer
2: Outputs:
   • a dense_layers* data structure containing the compressed weights matrix of each dense layer
   • a conv_layers* data structure containing the compressed weights matrix of each convolutional layer
3: set the number of centroids  $n_c$  according to Eq. (1)
4:  $dense\_layers^* \leftarrow dense\_layers$ 
5:  $conv\_layers^* \leftarrow conv\_layers$ 
6: for  $W_L \in dense\_layers$  do
7:    $C_{Dense}^{(L)} \leftarrow KM(W_L, n_c)$ 
8:   for  $w \in W_L$  do
9:      $w^* \leftarrow \arg \min \|C_{Dense}^{(L)} - w\|$ 
10:    update  $w$  in  $dense\_layers^*$  with  $w^*$ 
11:   end for
12: end for
13:  $C_{Conv} \leftarrow KM(conv\_layers, 256)$ 
14: for  $W_L \in conv\_layers$  do
15:   for  $w \in W_L$  do
16:      $w^* \leftarrow \arg \min \|C_{Conv} - w\|$ 
17:     update  $w$  in  $conv\_layers^*$  with  $w^*$ 
18:   end for
19: end for
20: return  $dense\_layers^*, conv\_layers^*$ 

```

where $\|W^{(max)}\|$ is the largest weight matrix among the dense layers, n_c is the number of centroids to be found, and d is 1 since the algorithm analyzes each weight as a single number $\in \mathbb{R}$.

Unlike the previous case, the algorithm performs the KM clustering only once (lines 13–19), in this sense, according to Eq. (3) the time complexity is $O(\|conv_layers\|^{(256)})$ where $\|conv_layers\|$ is the number of convolutional weights, 256 is the fixed number of centroids, and d is also in this case 1. By combining these two terms, the total algorithm complexity is:

$$O(\|W^{(max)}\|^{(n_c)}) + O(\|conv_layers\|^{(256)}) \quad (4)$$

however, if we consider that in a CNN the majority of the weights is concentrated in the dense layers, these makes the second term negligible in most of the neural network topologies.

5. Experimental results

This section provides a quantitative analysis of experimental results and comparisons of Deep Leaf 2.0. In particular, we show the results obtained from testing the new CNN we designed by comparing it with the previous version of our detector. To prove the effectiveness of the proposed compression algorithm both in terms of accuracy performance and memory compression, we compare it with the method available on X-CUBE-AI and the 8 bit integer quantization provided by TensorFlow Lite, showing also its applicability to different contexts domains.

5.1. CNN results

The main requirement we posed during the CNN design process is represented by the accuracy performance during the disease detection. Such a requirement is fundamental to implement a system with a high level of confidence during the diagnosis process.

Fig. 8 depicts the loss and accuracy plots of the CNN described in Section 3. With respect to the loss (shown in Fig. 8(a)) both the training and validation curves exhibit a decreasing trend which converges to the almost the same value demonstrating the correct learning of the deep learning model. Such a result is also confirmed by the plot we show in Fig. 8(b) where both the accuracy curves tend to increase at each epoch converging to very high level of accuracy around 0.95 (i.e., the 95%). In both cases the plots also show the early stopping point reached after 32 epochs which terminates the training before the 1000 epochs we set as maximum limit, thus avoiding the network overfitting. To further test the accuracy performance of the CNN, we computed the confusion matrix on the test set and extracted the precision, recall,

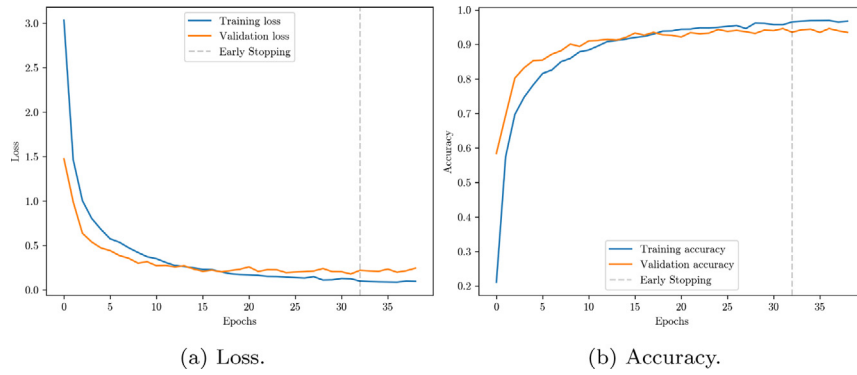


Fig. 8. Loss and accuracy plots of the proposed CNN model.

Table 4

Comparison between the two versions of the detector.

Metrics	Deep Leaf 2.0	Deep Leaf [7]
Precision	0.936	0.864
Recall	0.928	0.830
F1-score	0.931	0.831
Accuracy	0.95	0.91

Table 5

Performance indices.

Layer	Float 32 bit model	Quantized model	Compression in X-CUBE-AI ^a		Proposed compression	
			x4	x8	x4	x8
Precision	0.936	0.936	0.936	0.936	0.936	0.935
Recall	0.928	0.927	0.926	0.927	0.928	0.928
F1-score	0.931	0.931	0.929	0.931	0.931	0.931
Accuracy	0.95	0.95	0.948	0.95	0.95	0.95

^aSTM32 X-CUBE-AI – version 5.2.0.

and F1-score indices in order to have a better understanding of its confidence level during the inference phase. Moreover, to put in evidence the improvements made by Deep Leaf 2.0, we show the results extracted from the confusion matrix of the Deep Leaf [7] CNN tested on the Plant Village dataset described in Section 3. In Table 4 we report the indices extracted from both the CNNs. In general, the Deep Leaf [7] model reached acceptable results with values ranging from 0.864 to 0.91 which demonstrate sufficient generalization capabilities. If we consider that the initial version of our detector it has been designed to accomplish a simpler task, the obtained results were still good even if they did not satisfy the high accuracy requirements we posed. On the other hand, the results obtained from Deep Leaf 2.0 are very good with values of 0.936, 0.928, 0.931, and 0.95 for precision, recall, F1-score, and accuracy respectively, which demonstrate the higher level of generalization reached by the CNN model if compared to its previous version.

5.2. Compression algorithm results

In this subsection we present the results obtained from testing different types of compression algorithms, with the goal of understanding how these procedures impact the accuracy performance of the uncompressed 32 bit float model. To do that, we compared the proposed dynamic compression algorithm with 4 different model versions, namely: the 32 bit original floating point, the KM compressed models (i.e., x4 and x8) using the X-CUBE-AI tool, and a 8 bit quantized model using the TensorFlow lite framework. It is worth to mention that the experimental results have been obtained by testing each model with the same test set indicated in Section 3 to perform a fair comparison.

Table 5 shows the performance indices extracted from the test set confusion matrix. To allow a better understanding, we report also in this case the averaged precision, recall, and F1-score results computed over the 38 classes of the dataset. In general, the performance of the compression and quantization algorithms are very good reaching almost the same values of the original 32 bit floating point model. In particular, all the indices range between 0.927 and 0.95, thereby demonstrating the high confidence level of our disease detector in recognizing the health state of a crop even if compressed or quantized.

Such results also demonstrate the effectiveness of these methods in saving the original model performance, while reducing the model footprint such that it can fit the hardware restrictions of an Edge device.

Table 6
Compression results.

Layer	Float 32 bit model	Quantized model	Compression in X-CUBE-AI ^a		Proposed compression	
			x4	x8	x4	x8
conv1 (B)	9,728	2,528	9,728	9,728	2,528	2,528
conv2 (B)	36,992	9,344	36,992	36,992	9,344	9,344
conv3 (B)	73,984	18,688	73,984	73,984	18,688	18,688
conv4 (B)	147,712	37,120	147,712	147,712	37,120	37,120
LUT conv layers (B)	–	–	–	–	1,024	1,024
dense1 (B)	2,097,664	524,800	524,800	262,656	524,800	262,656
LUT dense1 (B)	–	–	1,024	64	1,024	64
dense2 (B)	66,048	16,896	16,896	8,704	16,896	8,704
LUT dense2 (B)	–	–	1,024	64	1,024	64
dense3 (B)	19,608	5,016	19,608	19,608	19,608	19,608
Total size (B)	2,451,736	614,392	831,768	559,512	632,056	359,800
Compression factor	–	3.99	2.95	4.38	3.88	6.81

^aSTM32 X-CUBE-AI – version 5.2.0.

With respect to the model size reduction, in Table 6 we report a comparison between the algorithms in terms of compression performance. With respect to dense layers, the two compression approaches (i.e., the one available in X-CUBE-AI and the proposed one) work in the same way. Depending on the target-factor (i.e., x4 or x8) passed by the user, the algorithm performs a KM clustering on each layer and remaps the weights into a reduced number of points (i.e., the centroids found by the KM). Considering the method available in X-CUBE-AI, a x4 target-factor generates a LUT for each dense layer of 1,024 B (i.e., 4 B · 256 centroids), introducing a compression factor of 2.95 that produces an overall size reduction of about the 66%. With respect to a x8 target-factor, the compression reaches even higher results with a LUT for each dense layer of 64 B (i.e., 4 B · 16 centroids) and a compression factor of 4.38, resulting in a model size reduction of about 77%. In every case, the last dense layer is not compressed since it represents the output of the network that it is left untouched to prevent wrong model behaviors.

The quantization proposed by TensorFlow works in a different way respect to compression. Specifically, the algorithm performs a linear quantization, computing the maximum and the minimum values of the weights for each neural network layer and creating an interval. The interval is then divided into 256 steps in the range $[-127, 127]$, where this number derives from the 8 bit precision adopted by this technique such that $2^8 = 256$. According to the value of each weight, it is remapped into the corresponding “symbol” associated to the step in which it falls. In this sense, a real number r is represented as a function of its 8 bit integer quantized value q , and two quantization parameters, namely: the scale factor s and the zero point parameter Z_p : $r = (q - Z_p) \cdot s$. [23]. Unlike the compression, a quantized model does not use a LUT to store the weights indices, but it directly converts the weights into an 8 bit representation format achieving the model footprint reduction. In particular, such a technique allowed to reduce the dense layers size to the exact same values obtained for the x4 target-factor in both the compression algorithms except for the last layer (i.e., dense3) which is also quantized in this case, passing from 19,608 B to 5016 B.

Focusing our attention on the convolutions, the proposed dynamic compression algorithm is able to strongly reduce the amount of bytes necessary to represent a layer with a memory reduction ranging from 71.4% (in *conv1*) to 74.7% (in *conv2*) with a LUT of 1.024 B due to the use of a fixed number of 256 centroids. Moreover, because the algorithm merges convolutions layer data and entirely remaps it into 256 values, it achieves the exact same size reduction of quantization without the need of converting the weights to 8 bit integers. In terms of compression factor, the quantization method reached a value of 3.99, resulting in a slightly better compression than the proposed approach with a x4 target-factor. Considering a x8 target-factor our dynamic method outperforms even the quantization with an improvement of 70.67%. Compared to the X-CUBE-AI compression method on the other hand, the proposed approach reached an overall compression factor of 3.88 and 6.81 respectively, resulting in an improvement of 31.5% for the x4 target-factor and 55.47% for the x8 one.

Fig. 9 depicts a size comparison between the six models we compared, providing also a visual representation of the models footprint showed in Table 6. Here, we can notice the effectiveness of our proposed method over the one used in X-CUBE-AI and the 8 bit quantization approach available in TensorFlow. Given the original 32 bit floating point model with a size of about 2.5 MB, our algorithm reduces its footprint to 632 KB with a x4 target-factor and 360 KB with a target-factor of x8, thereby resulting in an overall size reduction of about 74.2% and 85.2% respectively. Compared to the compression used in X-CUBE-AI, our method is able to outperform it in both cases with a further size reduction of 24% and 35.69% with a target-factor of x4 and x8 respectively. With respect to quantization, the proposed method reached comparable results with a x4 target-factor. In this sense, it is worth to mention that despite the increment of the model size of about 18 KB, the use of a compression technique does not require the weights conversion to integers that in some cases (especially in small models) cause an inevitable drop of the accuracy performance [23]. On the other hand, considering a x8 target-factor our compression method is able to outperform also the quantization with a further size reduction of about 41.4%.

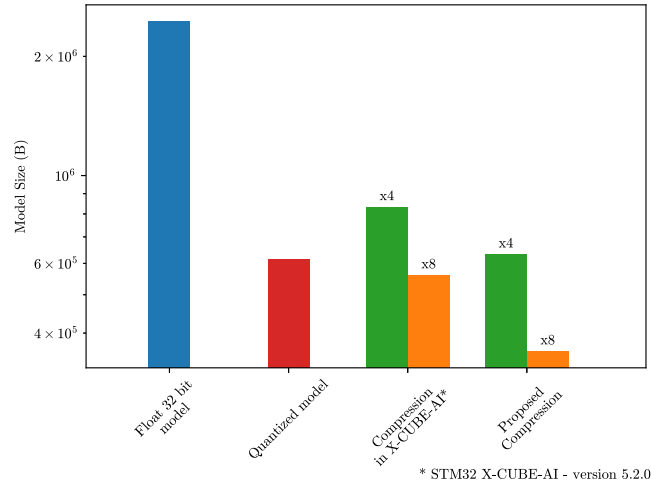


Fig. 9. Model sizes comparison.

In an era where the computation is shifting towards (constrained) Edge devices, the use of such algorithms is becoming increasingly common. The obtained results highlight the potential of compression techniques as a promising solution to reduce the complexity of a deep learning model while saving the original prediction performance and encourage the investigation of more sophisticated methods to further reduce the model memory footprint.

5.3. Energy consumption results

Another very important aspect when working with Edge devices is the one related to the energy consumption. In this sense, due to their battery driven nature, the software running on these should be designed to be energy efficient. The STM32CubeMX software allows to estimate the average energy consumption of a board depending on its setup. More precisely, by properly setting the board configuration in terms of clock frequency and active peripherals, it is possible to estimate the average current consumption. The peripherals enabled by our application include: Digital CaMera Interface (DCMI) to communicate with the camera module, Direct Memory Access (DMA) and Direct Memory Access 2D (DMA2D) for image manipulation, LCD-TFT Display Controller (LTDC) to control the display, and a timer (TM1). Furthermore, the external SDRAM and Quad Serial Peripheral Interface (QSPI) are necessary for communication.

Through this analysis our goal is to measure the average energy consumption for each one of the compression methods to quantitatively estimate the battery life of our ICPS. At the moment of writing, the proposed dynamic algorithm is still under review to be added in the future release of the X-CUBE-AI tool. Nevertheless, such an analysis allowed us to understand the behavior of these techniques, resulting very useful for the estimation of the performance of the proposed algorithm.

In this case, by taking in consideration the above mentioned components, the average absorbed current per millisecond (denoted as $\bar{i}$) was equal to 305.67 mA. Considering a supply voltage (V_{dd}) equal to 3.3 V and an average inference time ($\bar{t}$) of 12, 250.45 ms for the compressed models (measured using the X-CUBE-AI tool utilities), the average energy consumption ($\bar{E}$) for each inference is computed as:

$$\bar{E} = \bar{i} \cdot V_{dd} \cdot \bar{t}. \quad (5)$$

Thus, the two compressed models available in X-CUBE-AI (i.e., x4 and x8) have an average energy consumption of 12, 357.16 mJ for each inference. Considering a scenario like the one showed in Fig. 10(a) where one inference is performed every minute such that the device is active for about 13 s and then remains in an idle state for 47 s, if the board is powered with a configuration of 3 Ni-MH(F14000) batteries (arranged in series), each with a capacity of 14,000 mAh and a nominal voltage of 1.2 V, the expected lifetime of the batteries would be 8 days and 16 h.

Similarly, it is possible to obtain these results for the quantized model, whose inference time is on average equal to 4365.48 ms, thus consuming an average energy of 4, 393.87 mJ for each inference. In this sense, if the board is powered with the above described battery configuration, the expected lifetime of the batteries would be of 20 days and 17 h considering again a scenario in which one inference is performed per minute as shown in Fig. 10(b).

As one could expect, the quantization reached a lower value of energy consumption due to the 8 bit integer weight representation which makes faster the model operation with the consequent reduction of the inference time. Nevertheless, we have to think that quantization is substantially different from the compression, it requires a more careful hyperparameters tuning and can lead to significant accuracy loss [23]. According to the application context, such results allow the selection of best trade-off in terms of energy consumption and memory utilization.

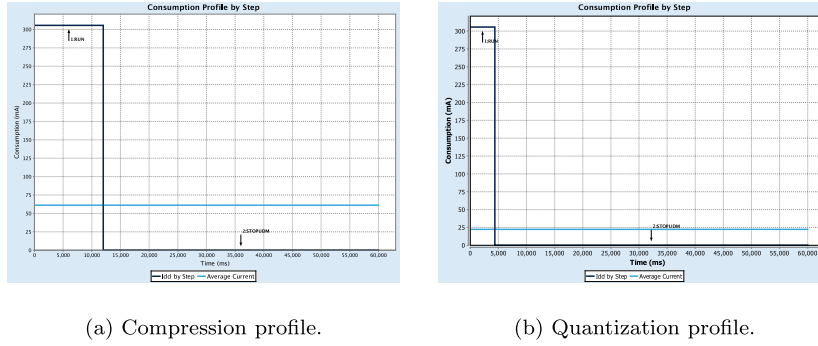


Fig. 10. Compression and quantization consumption profiles in one minute.

Table 7

Model energy consumption summary.

Model	Inference time (ms)	Energy consumption (mJ)	Battery lifetime (h)
Compression in X-CUBE-AI ^a	12,250	12,357	208
Quantized model	4365,48	4,393	497

^aSTM32 X-CUBE-AI – version 5.2.0.

Table 7 makes a summary of the energy consumption results obtained from testing both the two techniques. In such a context, we can notice that the quantization approach can benefit from the conversion of the weights into an 8 bit precision, strongly reducing the inference time and energy consumption of about 33% if compared to compression. If on the one hand the compression exhibits a larger energy consumption, on the other it allows to achieve a better compression. In this sense, if we consider that a battery life of 8 days and 16 h is already a very good result for an Edge device, the compression technique results to be the best trade-off in terms of model compression and energy consumption.

5.4. Other applications

To further prove the generality of the proposed dynamic compression method, we show its performance when applied to two different application context domains. With respect to the first one, we considered a Human Activity Recognition (HAR) system and we analyzed the results obtained from testing our compression algorithm. To build this application, we used the *Actitracker* dataset [27] released by the Wireless Sensor Data Mining (WISDM) lab, and containing data retrieved from the 3D accelerometer sensors embedded in the user smartphones. The raw data consists of 2980,765 samples distributed over 6 classes representing human activities, namely: *Walking*, *Jogging*, *Stairs*, *Sitting*, *Standing*, and *Lying Down*, that we divided into train, validation, and test sets using the 80%, the 10%, and the 10% respectively. In order to realize the HAR, we designed also in this case a CNN to perform a supervised classification using as input the time series data coming from the three-axis accelerometer. The CNN consists of 4 convolution layers with 32, 32, 64, and 64 filters respectively, a 2×2 kernel size, and strides equal to 2. To reduce the number of parameters, we adopted a max pooling layer with a pool size of 2×2 after each convolution. With respect to the classifier part, we used one dense layer with 64 neurons and a softmax layer with 6 neurons as the output of the network. To prevent overfitting, also in this case we used the regularization technique by inserting a dropout layer with a drop rate of 0.2 between the two dense layers. Finally, we used Adam as optimizer with a learning rate of 0.001.

In **Table 8**, we report the results extracted computing the confusion matrix of the above described CNN on the test set. The results, are in general good with values ranging from 0.90 to 0.924, but most importantly they are almost the same of the 32 bit float model demonstrating once again the capability of our compression method to maintain the same accuracy performance, also when compared with other methods (e.g., the X-CUBE-AI compression and TensorFlow Lite quantization).

Table 9 reports a comparison of the compression results. Likewise the results showed in **Table 6**, the proposed compression method and the one available in X-CUBE-AI work in the exact same way when applied to the dense layers. With regards to the convolution layers, our dynamic algorithm is able to remap the weights into 256 values achieving a size reduction equal to the one obtained through the 8 bit quantization. In terms of compression, our approach reached a value of 3.73 with a x4 target-factor reaching results comparable with the quantization and improving the compression of about 149% if compared with the algorithm available in X-CUBE-AI. Assuming a x8 target-factor, the obtained results are even better with an overall compression improvement of 22.5% and 191% respect to the quantization and X-CUBE-AI respectively.

With regards to the second application, we considered the Modified National Institute of Standards and Technology (MNIST) dataset which is one of the largest datasets for computer vision benchmarks [28]. The dataset is structured into

Table 8
HAR performance indices.

Layer	Float 32 bit model	Quantized model	Compression in X-CUBE-AI ^a		Proposed compression	
			x4	x8	x4	x8
Precision	0.90	0.90	0.90	0.90	0.90	0.90
Recall	0.916	0.915	0.916	0.914	0.915	0.913
F1-score	0.907	0.907	0.907	0.906	0.907	0.906
Accuracy	0.924	0.922	0.924	0.923	0.924	0.923

^aSTM32 X-CUBE-AI – version 5.2.0.**Table 9**
HAR compression results.

Layer	Float 32 bit model	Quantized model	Compression in X-CUBE-AI ^a		Proposed compression	
			x4	x8	x4	x8
conv1 (B)	640	256	640	640	256	256
conv2 (B)	16,512	4,224	16,512	16,512	4,224	4,224
conv3 (B)	33,024	8,448	33,024	33,024	8,448	8,448
conv4 (B)	65,792	16,640	65,792	65,792	16,640	16,640
LUT conv layers (B)	–	–	–	–	1,024	1,024
dense1 (B)	98,560	24,832	24,832	12,544	24,832	12,544
LUT dense1 (B)	–	–	1,024	64	1,024	64
dense2 (B)	1,560	408	1,560	1,560	1,560	1,560
Total size (B)	216,088	54,808	143,384	130,136	58,008	44,760
Compression factor	–	3.94	1.50	1.66	3.73	4.83

^aSTM32 X-CUBE-AI – version 5.2.0.**Table 10**
MNIST performance indices.

Layer	Float 32 bit model	Quantized model	Compression in X-CUBE-AI ^a		Proposed compression	
			x4	x8	x4	x8
Precision	0.9814	0.9815	0.9814	0.9811	0.9815	0.9812
Recall	0.9815	0.9814	0.9815	0.9811	0.9814	0.9812
F1-score	0.9814	0.9814	0.9814	0.9811	0.9814	0.9812
Accuracy	0.9815	0.9815	0.9815	0.9811	0.9815	0.9812

^aSTM32 X-CUBE-AI – version 5.2.0.

two sets, namely: training and test, which contain 60,000 and 10,000 28×28 grayscale pixel images of handwritten digits grouped into 10 classes (from 0 to 9). We built a supervised CNN composed by 3 convolution layers with 32, 64 and 128 filters, a 3×3 kernel size and strides equal to 3; to downsample the learnable features we used a single max pooling layer with a pool size of 2×2 . The classifier part on the other hand consists of a dense layer with 32 neurons and a softmax layer with 10 neurons as output of the network. Finally, to train the network we used the Adam optimizer with 0.001 as learning rate.

In Table 10 we report the results of the confusion matrix extracted from the test set of the dataset. The performance among the models remained the same with values ranging from 0.9811 to 0.9815 with minimal changes even for the compressed and quantized models proving the generalization capability of the proposed method.

Table 11 shows the comparison in terms of compression results between the proposed method, the one available in X-CUBE-AI, and the 8 bit integer quantization provided by TensorFlow Lite. Likewise the previous case, our approach remaps the weights of all convolution layers into 256 values, achieving the same performance of the quantization. Specifically, the proposed method achieved a 3.97 compression factor with a x4 target-factor making it perfectly comparable with the results obtained from the quantized model. Also in this case, the proposed method it has been able to outperform the quantization with a x8 target-factor reaching a compression value of 6.85 that produces an improvement of about 74.2%. Compared with the method available in the X-CUBE-AI tool, the proposed method is able to outperform it for both the x4 and x8 target factors with an improvement of about 46.5% and 82.8% respectively. Experimental results also in this case are very good and demonstrate the applicability of the proposed algorithm to different contexts domains.

Finally, to provide a better understanding of the proposed compression framework complexity, Table 12 reports the real execution time of the algorithm run on a machine equipped with an Intel Core i7 CPU, and 16 GB of RAM DDR4.

As we would expect, the running time strongly depends on the neural network topology. By analyzing the HAR model architecture it resulted to be the one with the lowest amount of convolution weights while having the largest amount of dense weights which explains why it has the largest running time with a x4 target-factor (i.e., 256 centroids). On the other hand, when working with a x8 target-factor (i.e., 16 centroids) the results are more comparable due to lower amount of centroids to be found by the KM algorithm.

Table 11
MNIST compression results.

Layer	Float 32 bit model	Quantized model	Compression in X-CUBE-AI ^a		Proposed compression	
			x4	x8	x4	x8
conv1 (B)	1,280	416	1,280	1,280	416	416
conv2 (B)	73,984	18,688	73,984	73,984	18,688	18,688
conv3 (B)	295,424	74,240	295,424	295,424	74,240	74,240
LUT conv layers (B)	–	–	–	–	1,024	1,024
dense1 (B)	1,982,592	495,744	495,744	247,936	495,744	247,936
LUT dense1 (B)	–	–	1,024	64	1,024	64
dense2 (B)	1,320	360	1,320	1,320	1,320	1,320
Total size (B)	2,354,600	589,448	868,776	620,008	592,456	343,688
Compression factor	–	3.99	2.71	3.80	3.97	6.85

^aSTM32 X-CUBE-AI – version 5.2.0.**Table 12**
Execution of the proposed compression algorithm.

Model	x4 running time (s)	x8 running time (s)
Deep Leaf	368.68	61.89
HAR	464.35	42.18
MNIST	323.14	69.05

6. Conclusions

In this paper, we presented a novel dynamic compression algorithm based on KM clustering to reduce the memory footprint of DNN models. Given a pre-trained model, our solution allows to fit the hardware constraints of an Edge device without the need of hyperparameters setup or model retraining. In this work we considered the case study of an ICPS application for the smart agriculture which is capable to detect a large number of diseases for several types of crops. Such an application is meant to be used as a support system for the human operator during the diagnosis process in plantation works to perform a fast early analysis of plants suspected to be infected. Experimental results demonstrated the effectiveness of the proposed compression method in reducing the memory footprint while saving the accuracy performance even when compared with other methods (e.g., the one available on the X-CUBE-AI tool, and a quantization technique used in TensorFlow Lite). Future works will be devoted to the improvement of Deep Leaf, enabling it to recognize more diseases, to the reduction of the energy consumption, and to the investigation of a method that can unify compression and quantization techniques to further improve the compression factor.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work of S. K. Das is partially supported by NSF grants SCC-1952045 (SIRAC project) and CNS-2008878 (FLINT project). The majority of the work was done while Fabrizio De Vita was visiting the Computer science Department at Missouri University of Science and Technology during 2019-2020.

References

- [1] P.K.R. Maddikunta, S. Hakak, M. Alazab, S. Bhattacharya, T.R. Gadekallu, W.Z. Khan, Q.V. Pham, Unmanned aerial vehicles in smart agriculture: Applications, requirements, and challenges, *IEEE Sens. J.* (2021) 1, <http://dx.doi.org/10.1109/JSEN.2021.3049471>.
- [2] R. Thapa, K. Zhang, N. Snaveley, S. Belongie, A. Khan, The Plant Pathology Challenge 2020 data set to classify foliar disease of apples, *Appl. Plant. Sci.* 8 (9) (2020) e11390.
- [3] R. Baheti, H. Gill, Cyber-physical systems, *Impact Control Technol.* 12 (1) (2011) 161–166.
- [4] P. Bellavista, et al., A survey on fog computing for the Internet of Things, *Pervasive Mob. Comput.* 52 (2019) 71–99.
- [5] D. Cogliati, M. Falchetto, D. Pau, M. Roveri, G. Viscardi, Intelligent cyber-physical systems for industry 4.0, in: 2018 First International Conference on Artificial Intelligence for Industries, *AI4I*, 2018, pp. 19–22, <http://dx.doi.org/10.1109/AI4I.2018.8665681>.
- [6] L. Lai, N. Suda, V. Chandra, CMSIS-NN: Efficient neural network kernels for arm cortex-M CPUs, 2018, *ArXiv preprint*, [arXiv:1801.06601](https://arxiv.org/abs/1801.06601).
- [7] F. De Vita, G. Nocera, D. Bruneo, V. Tomaselli, D. Giacalone, S.K. Das, Quantitative analysis of deep leaf: a plant disease detector on the smart edge, in: 2020 IEEE International Conference on Smart Computing, *SMARTCOMP*, 2020, pp. 49–56, <http://dx.doi.org/10.1109/SMARTCOMP50058.2020.00027>.
- [8] STM32Cube.AI, 2020, https://www.st.com/content/st_com/en/products/embedded-software/mcu-mpu-embedded-software/stm32-embedded-software/stm32cube-expansion-packages/x-cube-ai.html.

- [9] H. Li, K. Ota, M. Dong, Learning IoT in edge: Deep learning for the Internet of Things with edge computing, *IEEE Netw.* 32 (1) (2018) 96–101, <http://dx.doi.org/10.1109/MNET.2018.1700202>.
- [10] F. Wang, M. Zhang, X. Wang, X. Ma, J. Liu, Deep learning for edge computing applications: A state-of-the-art survey, *IEEE Access* 8 (2020) 58322–58336, <http://dx.doi.org/10.1109/ACCESS.2020.2982411>.
- [11] S. Kumar, B.K. Raghavendra, Diseases detection of various plant leaf using image processing techniques: A review, in: 5th Int’L Conf. on Advanced Computing Communication Systems, 2019, pp. 313–316, <http://dx.doi.org/10.1109/ICACCS.2019.8728325>.
- [12] K.P. Ferentinos, Deep learning models for plant disease detection and diagnosis, *Comput. Electron. Agric.* 145 (2018) 311–318, <http://dx.doi.org/10.1016/j.compag.2018.01.009>, <http://www.sciencedirect.com/science/article/pii/S0168169917311742>.
- [13] M. Hasan, M.S. Ahamed, A. Rakshit, K.M.Z. Hasan, Recognition of jute diseases by leaf image classification using convolutional neural network, in: 2019 10th International Conference on Computing, Communication and Networking Technologies, ICCNT, 2019, pp. 1–5, <http://dx.doi.org/10.1109/ICCNT45670.2019.8944907>.
- [14] S. Ramesh, et al., Plant disease detection using machine learning, in: 2018 International Conference on Design Innovations for 3Cs Compute Communicate Control, ICDI3C, 2018, pp. 41–45, <http://dx.doi.org/10.1109/ICDI3C.2018.00017>.
- [15] J. Luo, et al., ThiNet: A filter level pruning method for deep neural network compression, in: IEEE International Conference on Computer Vision, 2017, pp. 5068–5076, <http://dx.doi.org/10.1109/ICCV.2017.541>.
- [16] W. Chen, J. Wilson, S. Tyree, K. Weinberger, Y. Chen, Compressing neural networks with the hashing trick, in: International Conference on Machine Learning, 2015, pp. 2285–2294.
- [17] J. Wu, Y. Wang, Z. Wu, Z. Wang, A. Veeraraghavan, Y. Lin, Deep k-means: Re-training and parameter sharing with harder cluster assignments for compressing deep convolutions, in: J. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 80, PMLR, Stockholm, Sweden, 2018, pp. 5363–5372, <http://proceedings.mlr.press/v80/wu18h.html>.
- [18] D.P. Hughes, M. Salath'e, An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing, *CoRR abs/1511.08060* (2015) [arXiv:1511.08060](https://arxiv.org/abs/1511.08060).
- [19] TensorFlow Datasets, A collection of ready-to-use datasets, 2020, <https://www.tensorflow.org/datasets>.
- [20] A. Krizhevsky, et al., Imagenet classification with deep convolutional neural networks, in: Advances in Neural Information Processing Systems, 2012, pp. 1097–1105.
- [21] N. Srivastava, et al., Dropout: A simple way to prevent neural networks from overfitting, *J. Mach. Learn. Res.* 15 (1) (2014) 1929–1958.
- [22] K. Ullrich, E. Meeds, M. Welling, Soft weight-sharing for neural network compression, 2017, [arXiv:1702.04008](https://arxiv.org/abs/1702.04008).
- [23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, 2017, [arXiv:1712.05877](https://arxiv.org/abs/1712.05877).
- [24] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, <http://www.deeplearningbook.org>.
- [25] A. Vattani, K-means requires exponentially many iterations even in the plane, in: Proceedings of the Twenty-Fifth Annual Symposium on Computational Geometry, SCG '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 324–332, <http://dx.doi.org/10.1145/1542362.1542419>.
- [26] M. Inaba, N. Katoh, H. Imai, Variance-based k-clustering algorithms by voronoi diagrams and randomization, *IEICE Trans. Inform. Syst.* 83 (6) (2000) 1199–1206.
- [27] J.R. Kwapisz, G.M. Weiss, S.A. Moore, Activity recognition using cell phone accelerometers, *ACM SigKDD Explor. Newslett.* 12 (2) (2011) 74–82.
- [28] Y. LeCun, C. Cortes, C. Burges, MNIST Handwritten digit database, 2010, ATT Labs (Online), Available: <http://yann.lecun.com/exdb/mnist> 2.